26uam304

SLE-4: FINAL VIVA & ARCHITECTURE DECISION RECORD (ADR)

PART A: Architecture Decision Record (ADR)

Title: Keep BFS and DFS as Separate Search Functions

1. Context

My project is a Graph Search System that searches for a goal node from a start node. In SLE-2, I compared Breadth First Search (BFS) and Depth First Search (DFS) using the same graph. The program counted nodes expanded and measured execution time. In SLE-3, I organized the system into logical parts such as graph data, search engine, visited tracking, and output. I needed a design that makes the two algorithms easy to test, compare, and explain.

2. Decision

I decided to keep BFS and DFS as two separate functions inside the Search Engine. Both functions use the same graph data and return a path, but each uses its own search data structure.

3. Alternatives Considered

- Alternative 1 — Use only BFS: simple and finds a shortest path in an unweighted graph, but does not allow a direct DFS comparison.
- Alternative 2 — Use only DFS: can explore fewer nodes on some graphs, but does not guarantee the shortest path.
- Alternative 3 — Combine both algorithms into one complicated function: may reduce repeated code, but can make the logic harder for a beginner to understand and test.

4. Rationale / Justification

The SLE-2 profiling run recorded 7 nodes expanded by BFS and 4 by DFS on the selected graph. The recorded execution times were very small: BFS runs were approximately 0.010300 ms, 0.006700 ms, and 0.007100 ms; DFS runs were approximately

0.013800 ms, 0.004400 ms, and 0.004200 ms. These timings can change between runs and are too small to prove that one algorithm is always faster. The node counts show that DFS explored fewer nodes for this particular graph, while BFS and DFS follow different exploration strategies. Keeping them as separate functions makes the comparison fair because both use the same graph.

It also makes the code easier to read, debug, and profile. In the SLE-3 C4 design, both functions belong to the Search Engine, while the graph data, visited tracking, and output have clear responsibilities.

This design supports learning and makes it easier to add another search algorithm later.

5. Consequences

Positive:

BFS and DFS can be tested and profiled independently.

- The code structure is simple and maps clearly to the SLE-3 architecture.
- The same graph can be used for a fair comparison of nodes expanded and execution time.

Negative / Trade-offs:

- Some code is repeated in the two functions.
- Results depend on graph structure and neighbour order; one algorithm is not best for every problem.
- Execution-time measurements on a very small graph may be affected by measurement overhead.

6. References

SLE-2: BFS vs DFS profiling code and comparison results. SLE-3: Graph Search System using BFS and DFS — C4 Context, Container, Component, and Code overview. GitHub link: Add your repository URL

26uam304

PART B: Short Summary of My SLE Journey

In SLE-1, I worked on an AI/search-related Python program and prepared a README and AI Contribution Log. In SLE-2, I compared BFS and DFS using nodes expanded and execution time. My recorded run expanded 7 nodes with BFS and 4 with DFS, although execution times varied between runs. In SLE-3, I represented the Graph Search System using all four C4 levels: Context, Container, Component, and Code. For SLE-4, I recorded the decision to keep BFS and DFS as separate functions so that their behaviour can be tested and compared clearly.

PART C: AI Contribution Note

AI tools used: ChatGPT.

What AI helped with: Understanding the ADR format, organizing the decision and alternatives, and preparing clear explanations for the profiling results and C4 architecture.

What I did myself: I reviewed the project code, studied the BFS and DFS logic, used the available profiling results, and prepared to explain the architecture and design decision. I will verify the details against my actual code and report before submission.

PART D: Viva Preparation Notes

- **1. What is BFS?** BFS explores nodes level by level and uses a queue.
- **2. What is DFS?** DFS explores one branch deeply before backtracking and uses a stack.
- **3. Why keep separate functions?** It makes both algorithms easier to understand, test, and compare on the same graph.
- **4. What did SLE-2 show?** In my recorded run, BFS expanded 7 nodes and DFS expanded 4. Execution times were very small and varied between runs.
- **5. What is the purpose of a visited set?** It tracks explored nodes and helps prevent repeated processing and loops.
- **6. What is the C4 Model?** It describes software architecture at Context, Container, Component, and Code levels.

Reminder: Be ready to explain the code and results in your own words. Verify that the profiling values above match the final SLE-2 report you submitted.